



Thanks for buying SSSMapGenerator!

If you liked the asset, I would really appreciate it if you could leave a review.
If possible, a positive one.

If you have any comments, questions, suggestions for improvements,
feature requests, bug reports, or anything else, please feel free to contact me at:

E-mail : sw2.26tk@gmail.com
Twitter : https://twitter.com/sowa_jp

I'm Japanese, so I understand Japanese well. If you write in English, please use simple English.

▼About SSSMapGenerator

"SSSMapGenerator" is an asset that allows you to generate maps in the style of Slay the Spire.
It offers highly flexible customization, enabling you to create not only Slay the Spire-like maps,
but also unique maps based on your own original ideas.

As I am a Japanese developer, all program logic is fully commented in Japanese.

However, **for users in English-speaking regions, this asset includes a feature that converts the comments into English.**
Please run **"Tools > SSSMapGenerator > Japanese to English"** from the Unity menu.

▼Important Notes About SSSMapGenerator

This asset uses "TextMeshPro" and the "Input System", so please make sure they are installed in advance (required).

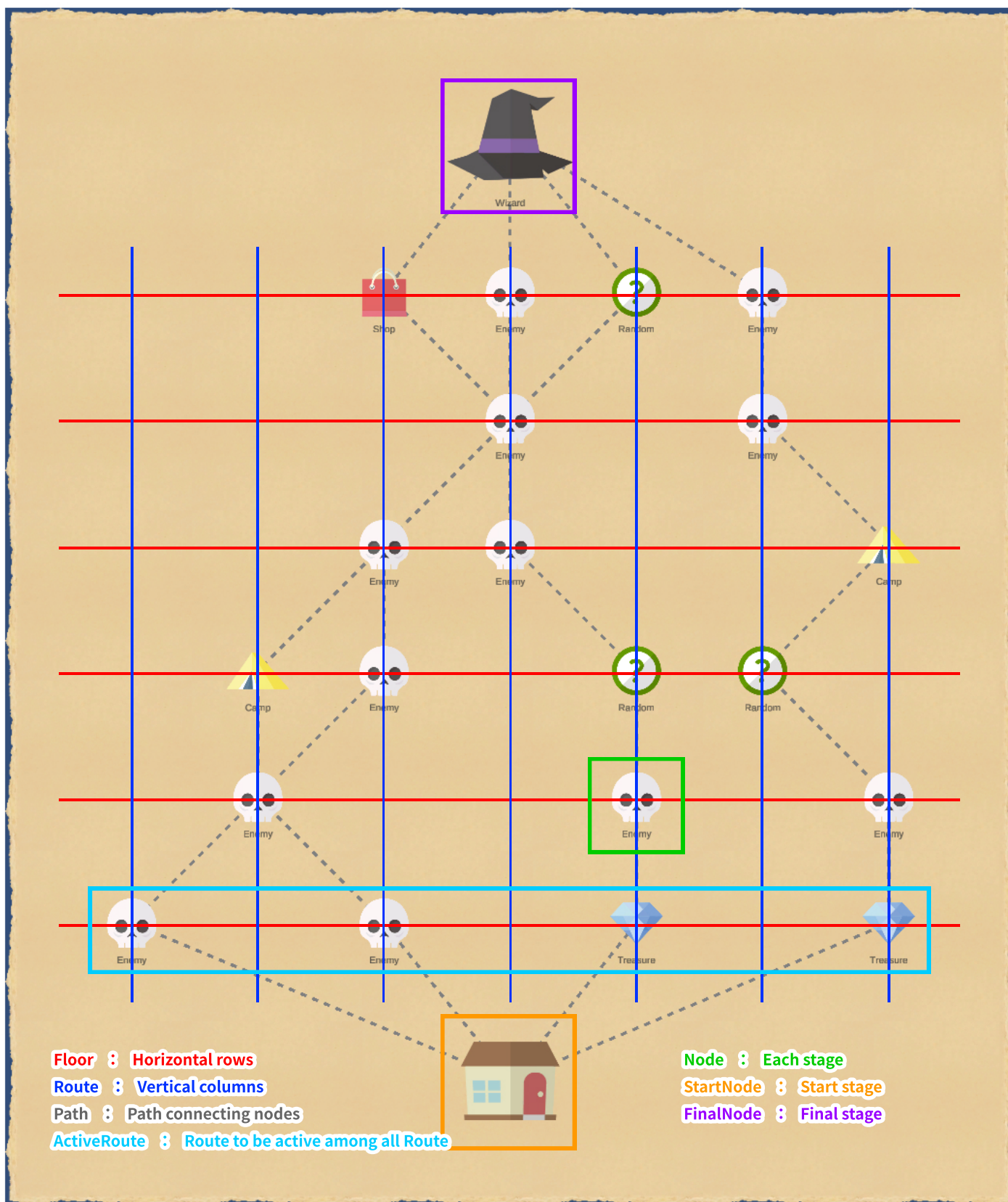
Implementing save/load functionality and handling node execution (such as enemies or shops) requires some scripting.

For this reason, this asset is not recommended for users who have no understanding of C# or who create games using visual scripting only.

The required code modifications are kept very simple.

Please refer to the Node Execution section for details on how each node is executed.

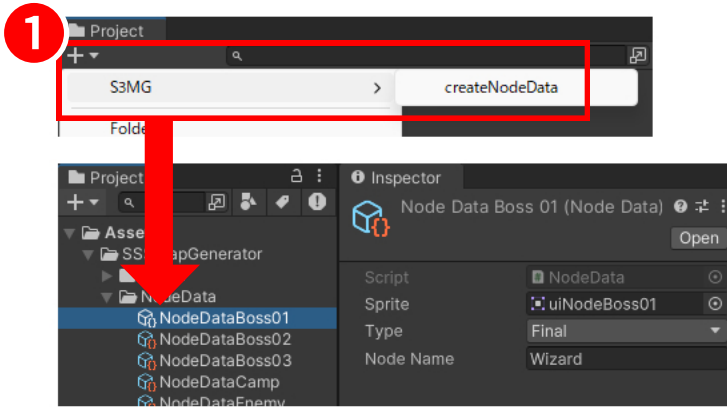
▼Basic



<Default Map Rules>

- Paths do not intersect: This can be changed in the settings.
- The same node type is not placed consecutively if possible: This can be changed in the settings.
However, depending on the settings, the rule may break, so at least one node type that can be placed consecutively must be registered.
If it still breaks, nodes will be assigned randomly.
- For nodes with branches, the destination must be a different type of node: This can be changed in the settings.
However, depending on the settings, the rule may break, so in such cases, nodes will be assigned randomly.
- If the map rules cannot be applied due to settings, after a certain number of attempts, the system will give up and assign nodes randomly to avoid generation failure.

▼ **Quick Start:** These steps describe how to set up from scratch. You may also use the Demo objects as-is.



Click

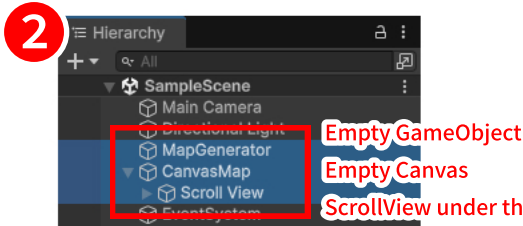
S3MG > createNodeData

in the Project window

and create as many node data assets as you like.

Each node data represents a stage.

Details about node data settings are explained on the next page, but the Node Name can be left empty.

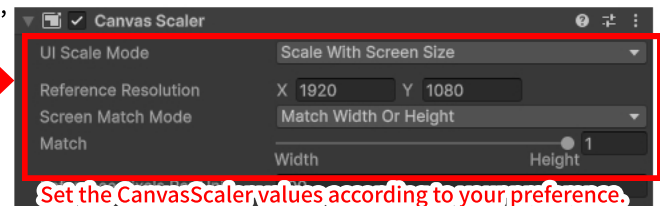


Create the following objects in the Hierarchy.

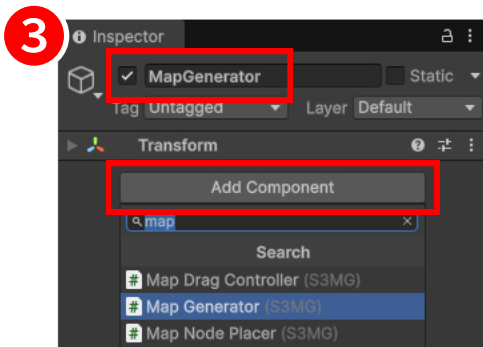
In the example on the left,

they are named:

- **MapGenerator**
- **CanvasMap**

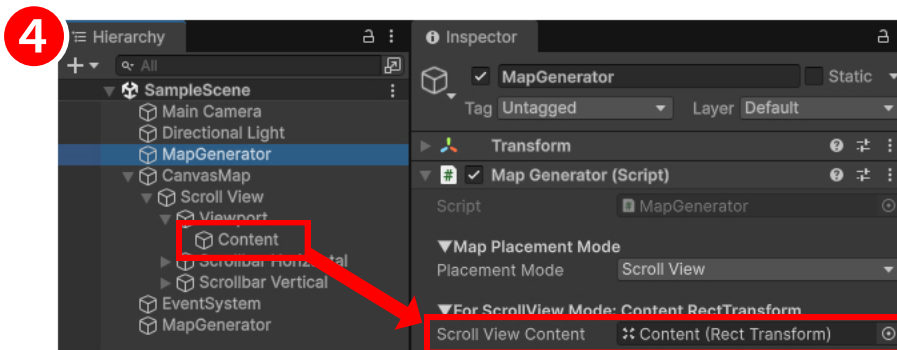


Set the CanvasScaler values according to your preference.



Add the following components to the empty GameObject you created:

- **MapGenerator.cs**
- **MapNodePlacer**
- **MapPathRenderer**
- **MapDragController**
- **MapValidator**
- **NodeEventHandler**
- **MapSaveManager**



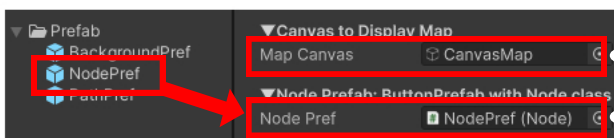
Set the required fields in the Inspector.

There are four required fields:

- **ScrollView Content**
- **Map Canvas**
- **Node Pref**
- **Node Data assets**

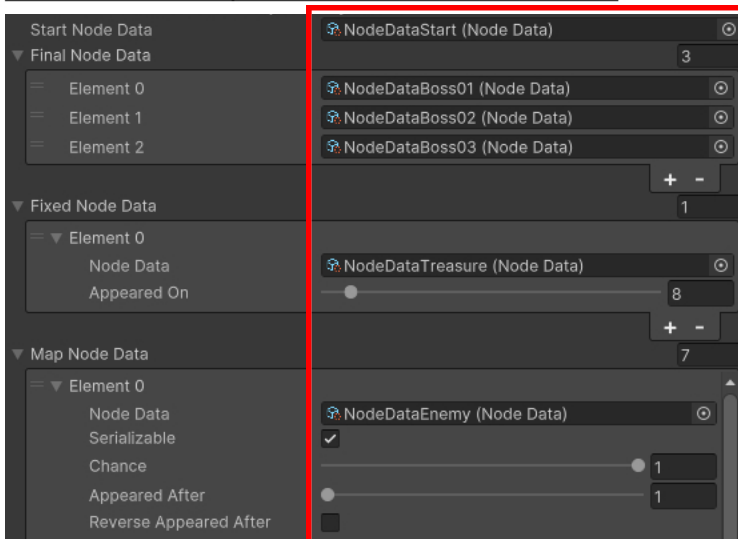
Other settings will be explained later.

← Content inside the ScrollView (not the ScrollView itself)



← The empty Canvas created in step 2

← Node Prefab in the Project window



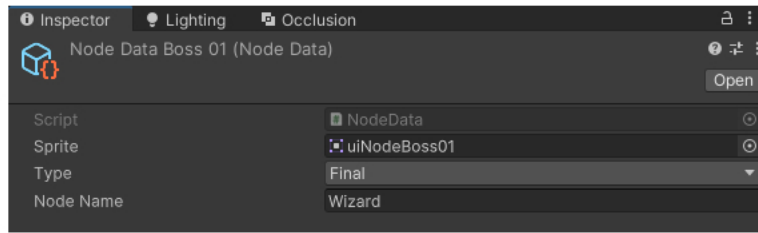
← Node Data assets created in step 1

Once this setup is complete, the map will be generated when you start the game.

Note:

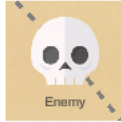
Component references are automatically assigned even if you do not attach them manually, but manual assignment is recommended for reliability.

▼Regarding createNodeData



<Sprite>

The image displayed on the node.
Please assign any square image you like.



<Type>

The node type.

By default, the following 11 types are available:

Empty, Start, Camp, Shop, Event, Treasure, Trap, Enemy, Middle, Final, Random

Please select any type you prefer.

If these are not sufficient, you can add your own custom types in the NodeData.cs class.

When adding a new type, you must also implement the corresponding logic in the NodeEventHandler.cs class.

For more details, see the “Extra” section under “How Each Node Is Executed”.

<Node Name>

The name of the node.

This field is optional, but if specified, the name can be displayed on the map.

Additionally, **even nodes with the same Type can be handled differently by assigning unique names.**

For example, if you create three nodes with the Type set to “Final” and name them “Warrior”, “Wizar”d, and “Ghost”, you can separate their behaviors in your code.

▼MapGenerator Settings



Mode Selection: Choose between Standalone Mode or ScrollView Mode

The Content object inside the ScrollView to be used in ScrollView Mode
★Required

If true, node processing is skipped and only the stage is moved.

Enable this option if you only want to test stage transitions.

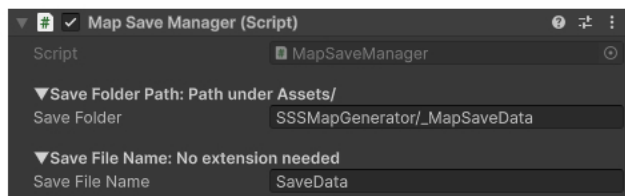
Randomize Seed: If true, the value of the seed variable is ignored.
Seed: Used when Randomize Seed is false.

Active Route Num:
If this value is greater than Route Num, it is limited to Route Num.
Map Canvas ★Required

Node Pref: Button prefab with the Node class. ★Required

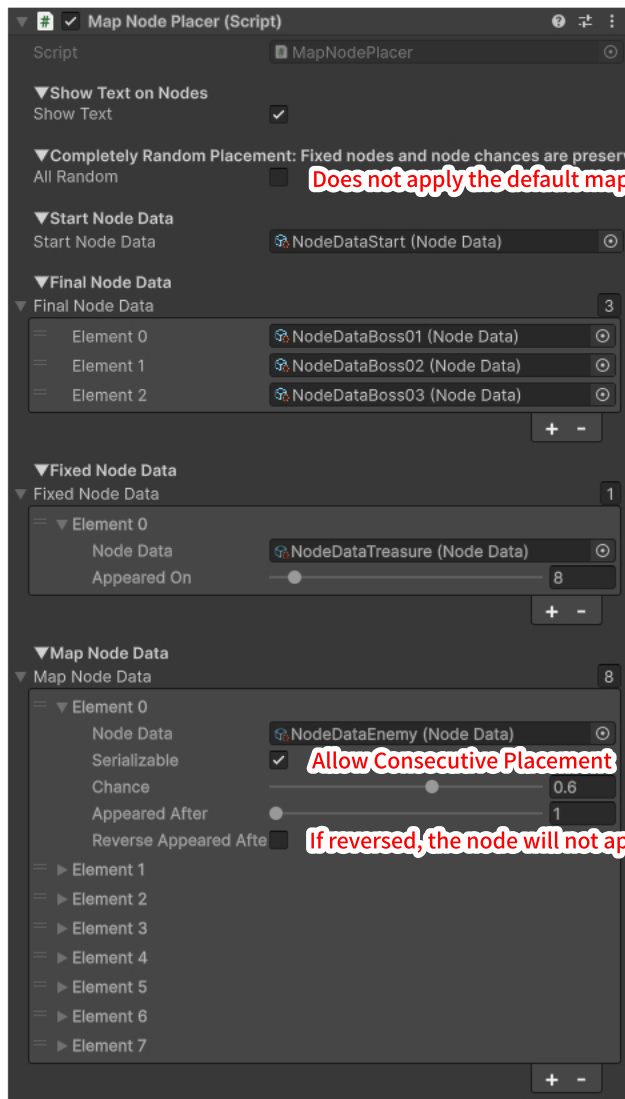
Background Pref: Image prefab with 9-slicing applied. Optional.

▼MapSaveManager



★Save and load functionality is explained in the “About Save and Load” section.

▼MapNodePlacer Settings



Does not apply the default map placement rules, except for fixed nodes.

Start Node Data ★Required when Create Start Node is true

Final Node Data ★At least one required
If multiple are assigned, one is selected at random.

Fixed Node Data
Multiple entries allowed. Optional.
Guaranteed to be created on the floor set in Appeared On.
If multiple entries have the same floor, the later one takes priority.

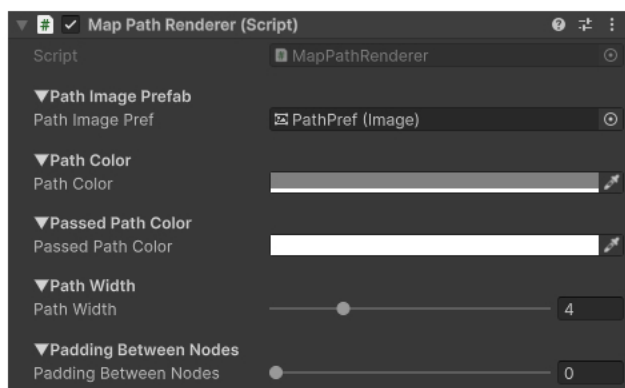
Map Node Data ★At least one required
multiple recommended

Allow Consecutive Placement

Spawn Probability

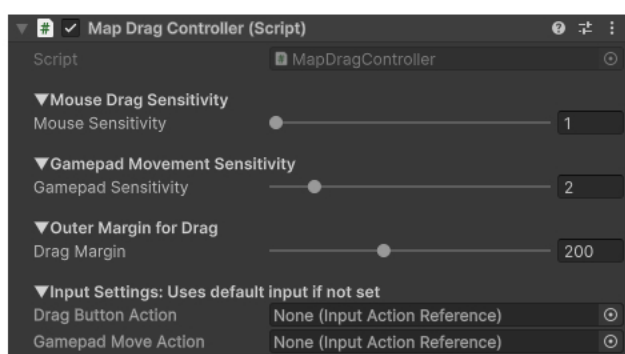
If reversed, the node will not appear on or after the floor set in Appeared After.

▼MapPathRenderer Settings



Custom prefabs with shaders can also be used. Optional.

▼MapDragController Settings



InputAction for Mouse Drag:
If not assigned, the left mouse button is applied automatically.
InputAction for Gamepad Movement:
If not assigned, the left stick of the gamepad is applied automatically.

▼How Each Node Is Executed

```
/*-----  
Processing for each node type: Edit here  
-----*/  
void OnStartNode(Node node){  
    Debug.Log($"Entered start node");  
}  
  
void OnCampNode(Node node){  
    Debug.Log($"Entered camp node");  
}  
  
void OnShopNode(Node node){  
    Debug.Log($"Entered shop node");  
}  
  
void OnEventNode(Node node){  
    Debug.Log($"Entered event node");  
}  
  
void OnTreasureNode(Node node){  
    Debug.Log($"Entered treasure node");  
}  
  
void OnTrapNode(Node node){  
    Debug.Log($"Entered trap node");  
}  
  
void OnEnemyNode(Node node){  
    Debug.Log($"Entered enemy node");  
}  
  
void OnMiddleNode(Node node){  
    Debug.Log($"Entered middle node");  
}  
  
void OnFinalNode(Node node){  
    Debug.Log($"Entered final node");  
    switch(node.NodeName){  
        case "Boss01":  
            Debug.Log($"Fighting Boss01");  
            break;  
        case "Boss02":  
            Debug.Log($"Fighting Boss02");  
            break;  
        default:  
            Debug.Log($"Fighting Boss03");  
            break;  
    }  
}  
  
void OnRandomNode(Node node){  
    Debug.Log($"Entered random node");  
}
```

When you open the NodeEventHandler.cs class, you will see the section shown in the figure on the left.

Functions corresponding to each node are prepared there, so feel free to write your own logic inside them.

As you can see in OnFinalNode(), even nodes with the same Type can execute different logic based on the Node Name.

<VERY IMPORTANT>

In each node's processing, there will be times when you want to move to the next node or show / hide the map. In such cases, call the following methods from the MapGenerator.cs class.

This class is implemented as a singleton, so please call these methods via instance.

- **ToNextNode()** Moves to the next node.
- **ActiveMap()** Displays the map.
- **InactiveMap()** Hides the map.
- **Init()** Generates the map manually when playOnAwake is set to false.
- **ReGenerate()** Rebuilds the map.

Ex: Hide the map in the destination scene
using S3MG;

```
public class SampleStartSceneManager : MonoBehaviour{  
  
    void Start(){  
        MapGenerator.instance.InactiveMap();  
    }  
  
}
```

<Extra>

As briefly mentioned on the “About createNodeData” page, you can also add custom node types.

To do so, first add your custom type to the enum in the NodeData.cs class.

Next, add the corresponding enum value to the switch statement

inside the “HandleNodeEntered()” function in the “NodeEventHandler.cs” class.

After that, simply define any function you want to call, just like the other node types, and you're good to go.

▼About Save and Load

Dedicated methods are provided in the MapSaveManager class.
Please call these methods to handle saving and loading.

<Ex: Calling from a Script>

using S3MG;

```
//Your own custom class
public class YourClass : MonoBehaviour{

    MapSaveManager saveManager;

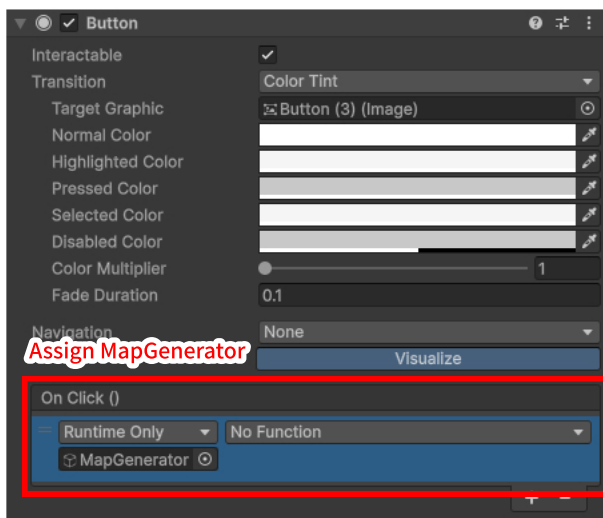
    void Start(){
        //Get MapSaveManager from MapGenerator.instance and cache it
        saveManager = MapGenerator.instance.GetComponent<MapSaveManager>();
    }

    void Save(){
        saveManager?.SaveMap();
    }

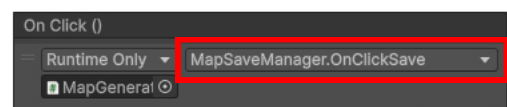
    void Load(){
        saveManager?.LoadMap();
    }

    void Delete(){
        saveManager?.DeleteSave();
    }
}
```

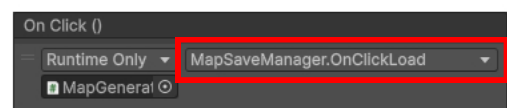
<Ex: Calling from a Button>



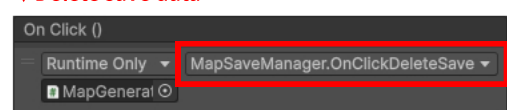
▼Save



▼Load



▼Delete save data



▼About each Prefab



Explanation of the Prefabs in the Prefab Folder

<BackgroundPref>

A prefab used for the background, containing only an Image component.

This prefab is assigned to “Background Pref” in MapGenerator, but its use is **optional**.

If no prefab is assigned, the background will display the Skybox or the color set on the camera.

The Image is configured with 9-slicing so that it expands according to the size of the map.

If you want to use a custom background, replace the Source Image of the Image component with a 9-sliced image.

<NodePref>

A prefab for nodes, containing a Button component, an AudioSource component, and the Node.cs class.

This prefab must be assigned to “Node Pref” in MapGenerator and is **required**.

You can freely change the Highlighted Color and Selected Color of the Button component.

The AudioClip of the AudioSource component is the sound played when the node is clicked and can be customized freely.

The Visit Image in the Node.cs class is a mark that appears on the node with an animation when it is clicked, and can be changed freely.

The Lerp Duration in Node.cs controls the blinking speed, and Lerp Color controls the blinking color; both can be customized.

<PathPref>

A prefab for paths, containing only an Image component.

This prefab is assigned to “Path Image Pref” in MapGenerator, but its use is **optional**.

If this prefab is assigned, dashed lines can be used.

If it is not assigned, simple solid lines will be drawn instead.

You can also assign your own custom images instead of dashed lines.

For example, you can use an image with a custom shader that animates the dashed lines.

In the demo, a shader that animates dashed lines is applied, so you can use it as is.

▼Release notes

For detailed information, please refer to the release log on the Asset Store.

▼Disclaimer

Please note that I cannot be held responsible for any issues or damages caused by the use of this asset.

▼Credit

This assets was developed by loloop (C) All rights reserved.